Data Structures-II

Session Objectives

- Understand what sets are.
- Understand common methods and operations associated with sets.
- Understand the comparison between lists, tuples and sets.
- Understand what dictionaries are.
- Understand common methods and operations associated with dictionaries.
- Understand the comparison between lists, tuples, sets and dictionaries.

Using Map() in Python:

What is Map?

- `Map(Function , Iterable)` where f(x) uses lambda function (One Linear Function)
- The Map() Function is a very handy tool in python that let's you:
  - Apply a Function to each items in a list, tuple or any other iterable
  - Returns a Map Object.

```
# Squaring each element
num_list = [1,2,3,4,5]
# map(Function , Iterable (num_list))
squared = map(lambda x : x ** 2 , num_list)
print(squared) # Map Object
```

```
<map object at 0x0000019A1BB62230>
```

```
# Any One of the Iterable will read the values of Map Objects -> It Prioritize based on who come first.
print(tuple(squared)) # (1,4,9,16,25)
```

```
(1, 4, 9, 16, 25)
```

```
print(list(squared)) # []
```

```
[]
```

```
# Adding 10 to the base num_list
num_list = [1,2,3,4,5]
# map(Function , Iterable (num_list))
add_10 = map(lambda x : x + 10 , num_list)
print(add_10) # Map Object
```

```
<map object at 0x0000019A1BBBBCA0>
```

```
print(list(add_10)) # [11,12,13,14,15]
```

```
[11, 12, 13, 14, 15]
```

```
print(tuple(add_10)) # ()
```

```
()
```

```
sir, I read somewhere that an input was taken using map(int(),input()).
What does it mean in terms of function and iterable?
```

```
# Taking an input from the user()
numbers = map(int, input("Enter the Val in a List Form: ").split())
print(numbers) # Map Object
```

```
Enter the Val in a List Form:  11 22 33 44 55
<map object at 0x0000019A1C7B2EF0>
```

```
print(list(numbers)) # [11,22,33,44,55]
```

```
[11, 22, 33, 44, 55]
```

```
print(tuple(numbers))
```

```
()
```

lambda F(x) -> One-Liner Function

Map(Function, Iterable)

[1,2,3,4,5]

Input

Lambda
F(X) => X^2

[1,4,9,16,25]

Output

```
for i in range(1,11)
```

iterate

Iterable [1,2,3,4,....10] [List]

```
def function_name(par1 , par2, ):
    function_body # Logic
```

DRY[Don't Repeat Yourself]

What are Sets? ¶

A Set in Python is :

1. `Unordered` : No guaranteed Order of Elements.
2. `Changeable (Mutable)` : You can add or remove elements.
3. `Unindexed` : There is no way to access an elements through Indexing & Slicing.
4. `Stoes Unique Element Stores` : Automatically remove duplicates.

*They can hold different **data types** all together, like numbers , strings and boolean*

```
_tuple = ()
_list = []
_set = {} ❌
_dict = {} ✅ Key-Value Pair
_set = set() ✅ Empty Set
```

```
_dict = {}
print(_dict)
print(type(_dict))
```

```
{}
<class 'dict'>
```

```python
_set = set()
print(_set)
print(type(_set))
```

```
set()
<class 'set'>
```

```python
_set = {11}
print(_set)
print(type(_set))
```

```
{11}
<class 'set'>
```

```python
_set = {1,2,3,'a','b','c',True,False,"Coding","Programming",99.99}
print(_set) # Mixed D-types , Unordered
```

```
{False, 1, 2, 3, 99.99, 'Programming', 'a', 'Coding', 'c', 'b'}
```

```python
_set[-1] # TypeError: 'set' object is not subscriptable
```

```python
_set[-5::2] # TypeError: 'set' object is not subscriptable
```

```python
_duplicate_set = {1,1,2,3,2,1,2,1,2,1,2,3,2,1,2,1,1} # {1,2,3}
print(_duplicate_set) # Unique Occurence # Unordered
```

```
{1, 2, 3}
```

```python
_duplicate_set = {1,1,2,3,2,1,2,1,2,1,2,3,2,1,2,1,1,'a','a','b','c','c','b'}
print(_duplicate_set)
```

```
{1, 2, 3, 'c', 'b', 'a'}
```

```python
_duplicate_set = {1,2,1,2,1,2,1,0,1,0,0,0,1,1,1,False,True}
print(_duplicate_set) # False [0] , True [1]
```

```
{0, 1, 2}
```

```python
_duplicate_set = {True,False,1,2,1,2,1,2,1,0,1,0,0,0,1,1,1,False,True}
print(_duplicate_set) # False [0] , True [1]
```

```
{False, True, 2}
```

```python
# set() constructor to do the typecasting.... (list,tuple)
country_tuple = ('India','America','Ukraine','New Zealand','Australia','South Africa',
                'Russia','Italy','Spain','Germany','Nepal','Ireland','Iran','Pakistan',
                'Canada','India','America','Germany','China','New Zealand','Australia','Singapore')
print(set(country_tuple))
```

```
{'America', 'Spain', 'Italy', 'Iran', 'Australia', 'India', 'South Africa', 'Nepal', 'Pakistan', 'China', 'Sin
gapore', 'Ukraine', 'Ireland', 'Canada', 'Russia', 'New Zealand', 'Germany'}
```

```python
_list = [False,True,0,1,"Coding",'Program','k',19.99,'a','c','d','a','a','a','c','k']
print(set(_list))
```

```
{False, True, 'k', 'a', 'Coding', 'Program', 19.99, 'd', 'c'}
```

```python
# Set can only Store Immutable Elements
nested_set = {
    'coding',
    99,
    ('a','b','c','d'),
    (('coding')),
    (('coding'),),
    (True,False)
}
nested_set
```

```
{('a', 'b', 'c', 'd'), ('coding',), (True, False), 99, 'coding'}
```

```python
# Understand common methods and operations associated with sets.
# Accessing an item is not allowed using Indexing
# 'in' operator [Membership Operator] -> Boolean Returns [T/F]
car_set = {'Taigun','Creata','Thar','Slavia','Scorpio','Hummer','Verna','Defender','Verna',
          'City','Safari','Harrier','Beatz','Cruze','Be6','Curvv','Ertiga','Kylaq'}
print('MiniCooper' in car_set) # False
print('Verna' in car_set) # True
print('Defender' in car_set) # True
print('Alto' in car_set) # False
```

```
False
True
True
False
```

```python
# length of a set -> will be counted after removing duplicates
car_set = {'Taigun','Creata','Thar','Slavia','Scorpio','Hummer','Verna','Defender','Verna',
          'City','Safari','Harrier','Beatz','Cruze','Be6','Curvv','Ertiga','Kylaq',
          'City','Safari','Harrier','Beatz','Cruze','Be6','Curvv','Ertiga','Kylaq'}
print(len(car_set))
```

```
17
```

```python
# min(), max() , sum() -> Continuous Column [Numerical Columns]
quick_set = {11,22,33,44,55,66,77,88,99,110,False,True,11,22,33,44,55} # 'mixed Dtype'
# int(False) = 0 & int(True) = 1
print(len(quick_set)) # 12
print(min(quick_set)) # False - 0
print(max(quick_set)) # 110
print(sum(quick_set)) # 606
```

```
12
False
110
606
```

```python
# min(), max() , sum() -> Continuous Column [Numerical Columns]
quick_set = {11,22,33,44,55,66,77,88,99,110,False,True,11,22,33,44,55,0,1} # 'mixed Dtype
# int(False) = 0 & int(True) = 1
print(len(quick_set)) # 12
print(min(quick_set)) # False - 0
print(max(quick_set)) # 110
print(sum(quick_set)) # 606
```

```
12
False
110
606
```

```python
# Adding items in a set()
# .add() to add a single element on a set.
quick_set = {11,22,33,44,55,66,77,88,99,110,False,True,11,22,33,44,55,0,1}
quick_set.add(99) # already exist
print(quick_set)
```

```
{False, 33, 66, 99, True, 11, 44, 77, 110, 22, 55, 88}
```

```python
quick_set.add(121) # will be added
print(quick_set)
```

```
{False, 33, 66, 99, True, 11, 44, 77, 110, 22, 55, 88, 121}
```

```python
quick_set.add("Coding") # will be added
print(quick_set)
```

```
{False, 33, 66, 99, True, 11, 44, 77, 110, 'Coding', 22, 55, 88, 121}
```

```python
quick_set.add("Python") # will be added
print(quick_set)
```

```
{False, 33, 66, 99, True, 'Python', 11, 44, 77, 110, 'Coding', 22, 55, 88, 121}
```

```python
# .update(<iterable>) -> Adds a multiple Elements -> Iterable as an argument
quick_set.update(['x','y','z'])
print(quick_set)
```

```
{False, 33, 66, 99, True, 'x', 'Python', 'z', 11, 44, 77, 110, 'Coding', 22, 55, 88, 121, 'y'}
```

```python
quick_set.update(('x','y','z'))
print(quick_set) # No Changes as elements already exists
```

```
{False, 33, 66, 99, True, 'x', 'Python', 'z', 11, 44, 77, 110, 'Coding', 22, 55, 88, 121, 'y'}
```

```python
# String insertion -> String is also an Iterable ✅
quick_set.update("Coding")
print(quick_set) # ['C','o','d','i','n','g']
```

```
{False, True, 11, 'g', 'd', 22, 33, 'C', 'z', 44, 55, 'n', 'x', 66, 77, 88, 99, 'o', 'Python', 110, 'i', 'Codi
ng', 121, 'y'}
```

```python
quick_set.update(("set",)) # tuple
print(quick_set)
```

```
{False, True, 11, 'g', 'd', 22, 33, 'C', 'z', 44, 55, 'n', 'x', 66, 77, 88, 'set', 99, 'o', 'Python', 110,
'i', 'Coding', 121, 'y'}
```

```python
quick_set.update(("coding",)) # tuple
print(quick_set)
```

```
{False, True, 11, 'g', 'd', 22, 33, 'C', 'z', 44, 55, 'n', 'x', 66, 77, 'coding', 88, 'set', 99, 'o', 'Pytho
n', 110, 'i', 'Coding', 121, 'y'}
```

```python
quick_set.update(("Python",)) # tuple
print(quick_set) # No Changes as "Python" as an element already exist
```

```
{False, True, 11, 'g', 'd', 22, 33, 'C', 'z', 44, 55, 'n', 'x', 66, 77, 'coding', 88, 'set', 99, 'o', 'Pytho
n', 110, 'i', 'Coding', 121, 'y'}
```

```python
# Removing an element from a set()
# .remove() -> throws an errors if the element doesn't exist
# .discard() -> it won't throws an errors even if the element doesn't exist
quick_set.remove("coding")
print(quick_set)
```

```
{False, True, 11, 'g', 'd', 22, 33, 'C', 'z', 44, 55, 'n', 'x', 66, 77, 88, 'set', 99, 'o', 'Python', 110,
'i', 'Coding', 121, 'y'}
```

```python
quick_set.remove("Star") # KeyError: 'Star'
print(quick_set)
```

```python
quick_set.remove(['c','o','d','i','n','g'])
print(quick_set)
# TypeError: unhashable type: 'list'
```

```python
quick_set.remove('C')
quick_set.remove('o')
quick_set.remove('d')
quick_set.remove('i')
quick_set.remove('n')
quick_set.remove('g')
print(quick_set)
```

```
{False, True, 11, 22, 33, 'z', 44, 55, 'x', 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121, 'y'}
```

Note : Set is a Mutable Container that only stores immutable elements.

```
quick_set.remove(['c','o','d','i','n','g'])
print(quic
```

```
Docstring:
Remove an element from a set; it must be a member.

If the element is not a member, raise a KeyError.
Type:      builtin_function_or_method
```

```
---------                                                    --------------
TypeError                                                    ent call last)
Cell In[4
----> 1 quick_set.remove([ ... ])
      2 print(quick_set)

TypeError: unhashable type: 'list'
```

Fix Code

```
quick_set.discard(('a','b','c')) # It won't throw an error even if the values doesn't exist
print(quick_set)
```

```
{False, True, 11, 22, 33, 'z', 44, 55, 'x', 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121, 'y'}
```

```
quick_set.discard(('x','y','z')) #  # It won't throw an error even if the values doesn't exist
print(quick_set)
```

```
{False, True, 11, 22, 33, 'z', 44, 55, 'x', 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121, 'y'}
```

```
quick_set.discard('x')
quick_set.discard('y')
quick_set.discard('z')
print(quick_set)
```

```
{False, True, 11, 22, 33, 44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```
quick_set.add(('x','y','z'))
print(quick_set)
```

```
{False, True, 11, 22, 33, ('x', 'y', 'z'), 44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```
quick_set.discard(('x','y','z'))
print(quick_set)
```

```
{False, True, 11, 22, 33, 44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```
# .pop() -> Removes and returns the popped element from an arbitary position. Since sets are unordered.
# You don't even know which items is popped()
popped_item = quick_set.pop()
print(popped_item) # False
print(quick_set) # False
```

```
False
{True, 11, 22, 33, 44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```
popped_item = quick_set.pop()
print(popped_item) # True
print(quick_set)
```

```
True
{11, 22, 33, 44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```
popped_item = quick_set.pop()
print(popped_item) # 11
print(quick_set)
```

```
11
{22, 33, 44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```
popped_item = quick_set.pop()
print(popped_item) # 22
print(quick_set)
```

```
22
{33, 44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```python
popped_item = quick_set.pop()
print(popped_item) # 33
print(quick_set)
```

```
33
{44, 55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```python
popped_item = quick_set.pop()
print(popped_item) # 44
print(quick_set)
```

```
44
{55, 66, 77, 88, 'set', 99, 'Python', 110, 'Coding', 121}
```

```python
day_set = {'Mon','Tue','Wed','Thurs','Fri','Sat','Sun'}
print(day_set)
pop_item = day_set.pop()
print(pop_item) # You can't predict
print(day_set)
```

```
{'Sun', 'Mon', 'Tue', 'Wed', 'Thurs', 'Sat', 'Fri'}
Sun
{'Mon', 'Tue', 'Wed', 'Thurs', 'Sat', 'Fri'}
```

```python
day_set = {'Mon','Tue','Wed','Thurs','Fri','Sat','Sun','Jan'}
print(day_set)
pop_item = day_set.pop()
print(pop_item) # You can't predict
print(day_set)
```

```
{'Jan', 'Mon', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
Jan
{'Mon', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Mon'
print(day_set)
```

```
Mon
{'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Thurs'
print(day_set)
```

```
Thurs
{'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
day_set = {'Mon','Tue','Wed','Thurs','Fri','Sat','Sun','Jan','Feb'}
print(day_set)
pop_item = day_set.pop()
print(pop_item) # You can't predict
print(day_set)
```

```
{'Feb', 'Jan', 'Mon', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
Feb
{'Jan', 'Mon', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Jan'
print(day_set)
```

```
Jan
{'Mon', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
day_set.add('Mar')
print(day_set)
```

```
{'Mar', 'Mon', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
day_set.add('Apr')
print(day_set)
```

```
{'Apr', 'Mar', 'Mon', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item)
print(day_set)
```

```
Mon
{'Apr', 'Mar', 'Thurs', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Thurs'
print(day_set)
```

```
Thurs
{'Apr', 'Mar', 'Sun', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Sun'
print(day_set)
```

```
Sun
{'Apr', 'Mar', 'Fri', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Fri'
print(day_set)
```

```
Fri
{'Apr', 'Mar', 'Tue', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Tue'
print(day_set)
```

```
Tue
{'Apr', 'Mar', 'Sat', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Sat'
print(day_set)
```

```
Sat
{'Apr', 'Mar', 'Wed'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Wed'
print(day_set)
```

```
Wed
{'Apr', 'Mar'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Apr'
print(day_set)
```

```
Apr
{'Mar'}
```

```python
pop_item = day_set.pop()
print(pop_item) # 'Mar'
print(day_set)
```

```
Mar
set()
```

```python
char_set = {'a','b','c','d','e','f','g','h'}
print(char_set) # Memory
pop_item = char_set.pop()
print(pop_item) # You can't predict
print(char_set)
```

```
{'e', 'f', 'g', 'a', 'h', 'd', 'c', 'b'}
e
{'f', 'g', 'a', 'h', 'd', 'c', 'b'}
```

```python
char_set = {'a','b','c','d','e','f','g','h',1,2,3,4,5,6,False,True}
print(char_set) # Memory
char_set.add("Coding")
print(char_set)
```

```
{'e', 1, 2, 3, 4, 'f', 5, 6, False, 'g', 'a', 'h', 'd', 'c', 'b'}
{'e', 1, 2, 3, 4, 'f', 5, 6, False, 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item) # 'e'
print(char_set)
```

```
e
{1, 2, 3, 4, 'f', 5, 6, False, 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item) # '1'
print(char_set)
```

```
1
{2, 3, 4, 'f', 5, 6, False, 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item) # '1'
print(char_set)
```

```
2
{3, 4, 'f', 5, 6, False, 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
char_set.add("Python")
print(char_set)
```

```
{3, 4, 'f', 5, 6, False, 'Python', 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
3
{4, 'f', 5, 6, False, 'Python', 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
f
{5, 6, False, 'Python', 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
5
{6, False, 'Python', 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
6
{False, 'Python', 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
False
{'Python', 'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
Python
{'g', 'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
g
{'a', 'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
a
{'h', 'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
h
{'Coding', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
Coding
{'d', 'c', 'b'}
```

```python
char_set.add("k")
print(char_set)
```

```
{'k', 'd', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
d
{'k', 'c', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
c
{'k', 'b'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
b
{'k'}
```

```python
pop_item = char_set.pop()
print(pop_item)
print(char_set)
```

```
k
set()
```